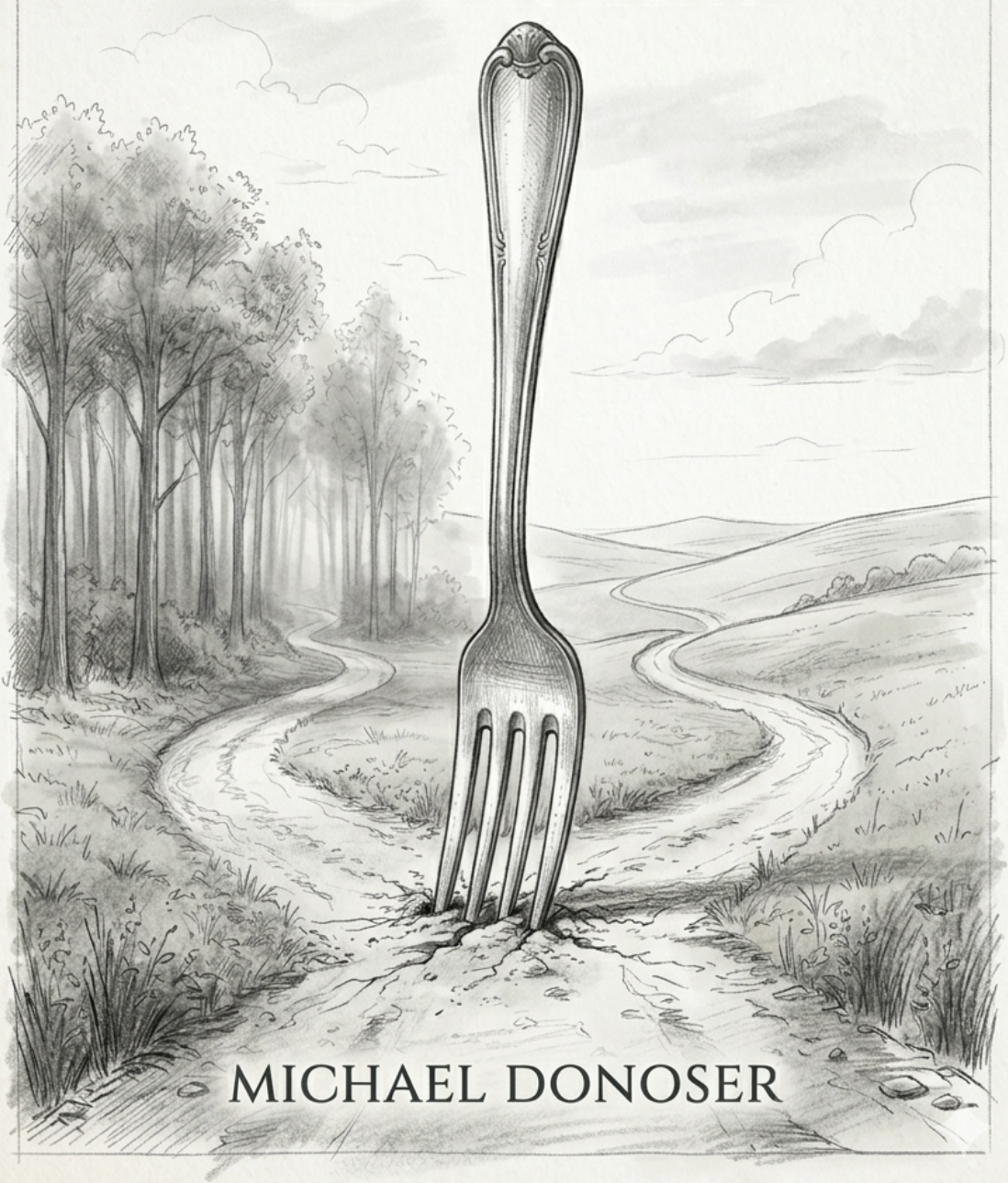
THE FORK IN THE ROAD

LEADERSHIP LESSONS FOR A WORLD WITH NO PLAYBOOK



MICHAEL DONOSER

About These Stories

Over eleven years at Amazon—leading cross-functional tech innovation teams building products that reached millions of customers—I made an extraordinary number of mistakes. I missed signals I should have caught. I ran meetings that should never have happened. I gave feedback that landed wrong, made decisions that looked reasonable at the time and embarrassingly stupid in hindsight, and occasionally led a room full of talented people directly into a wall.

What I also did, eventually, was pay attention. I learned from my mistakes, and I doubled down on a natural curiosity—a stubborn need to understand why things work the way they do —that, over time, turned those mistakes into something more useful than regret.

The question that consumed me more than any technical challenge was simpler and stranger: why do some teams seem almost effortlessly effective—energised, honest, driven, resilient under pressure—while others, staffed with equally talented people and similar resources, quietly struggle? I watched this play out repeatedly. Same company. Same tools. Same hiring bar. Completely different outcomes.

My working conclusion, validated over many years and countless situations, is that the difference almost never lives in the skillset. It lives in the culture—in the small, almost invisible choices that accumulate into habits defining how a team actually behaves and communicates, and especially how it holds together when things get hard. As they always do.

I spent a long time thinking about how to share what I had learned. Frameworks felt too clean, principles too abstract. I kept coming back to stories—because that is where these lessons actually live. So instead of laying them out as theory, I captured them as human situations

where the gap between intention and behaviour becomes clear. The stories are hypothetical—not transcripts of real events—but grounded in situations I witnessed, participated in, or caused. I've changed the details, compressed the timelines, and constructed composite characters. The underlying dynamics are real. I've lived most of them.

Each story follows the same structure: a situation any tech company might face, shown through two hypothetical teams—Team Alpha and Team Beta—with no meaningful difference in skill, experience, or intent. The difference is in how they behave, and in the specific mechanisms—repeatable, concrete practices—that produce that behaviour. At the end of each story, I describe those mechanisms directly. Not as values or aspirations, but as things you can start doing on Monday morning.

One more thing worth saying clearly: none of what follows is only for the boss, or for the most senior person in the room. Everyone is a leader. Every person on a team, regardless of title or tenure, can shape culture and influence how the work gets done. The behaviours in these stories are not reserved for those with authority— they are things anyone, at any level, can and should own.

I have no illusion that the ideas in these pages are all original—most of what I found useful I found elsewhere, in the work of people far more rigorous than I am. My contribution is to connect those ideas into something coherent and anchor them in situations where they actually matter. I got a lot wrong over the years. I'm sharing what I learned in the hope that some of it is useful to someone else.

This is the first story.

EXAMPLE CHAPTER

The Launch Disaster

How our two teams faced the same disaster and chose entirely different futures

The Champagne Was Already Chilled

The launch of the AI-driven feature code-named "Futurama" was supposed to be a masterpiece of innovation. Futurama was not a single-team project—it was a sprawling cross-organisational effort, drawing together groups that had rarely, if ever, worked closely with one another: machine learning researchers, platform engineers, product and design teams, data scientists, and stakeholders from several different parts of the business, each bringing their own priorities, processes, and ways of working.

For six gruelling months, this unlikely coalition had moved through the full arc of a major product effort. It began with a planning document that forced the group to agree, sentence by sentence, on exactly what they were promising their customers and why—before a single line of code was written.[1] Only once that document had survived its reviews did the real work begin: engineers writing and shipping code, scientists developing and tuning models, designers iterating on the experience. Running alongside it were weekly cross-org progress reviews, late-night debugging sessions, and the kind of methodical error hunts that usually only high-priority, customer-facing features get.

The release date had already slipped twice—not from lack of effort, but from the kind of compounding friction that complex, cross-team projects attract when they are being done carefully. By the time the date finally held, the teams were carrying the particular exhaustion of people who had been sprinting for too long. But underneath the tiredness was genuine excitement. Futurama was, by any honest assessment, a remarkable piece of work—the kind of feature that could substantially

[1] At Amazon, this is the *PR/FAQ*: a hypothetical press release describing the finished product from the customer's perspective, followed by anticipated questions on feasibility, cost, and risk.

change how customers experienced the product. Everyone involved had poured themselves into it—taking full ownership, going beyond what was asked, and pushing through the setbacks and the slippage—and could feel, in their bones, that everything that was about to happen had been earned.

Across the building, the contributing teams gathered in their own conference rooms—some had chilled champagne ready, others had laid out fancy finger food. Usage dashboards were projected on the screens at the front, everyone's eyes fixed on two numbers: feature adoption, expected to spike immediately as customers discovered what they had built, and sales, which the new feature was meant to drive meaningfully upward in the weeks that followed. The room was quiet. With a single click, the code went live.

Then came the eerie silence.

The numbers were moving in entirely the wrong direction. The new feature was nowhere to be found—but that was almost secondary. The deployment had broken something deeper: the existing pipeline that millions of customers relied on every day had started failing. Support tickets were already coming in. Customers who had never heard of Futurama were finding that tools they had used without incident for years were suddenly not working.

A massive bug had slipped through the final gate. Later, post-mortems would call it a "complex failure": the kind that doesn't emerge from a single mistake, but from a confluence of small, individually harmless decisions that, combined for the very first time under live conditions, produced catastrophe. The root cause was buried deep in the invisible seams between the responsibilities of multiple different teams. No single group had "messed up" in any obvious sense. The system itself had failed—and the system was all of them.

* * *

This was the fork in the road.

From here, the story splits. What follows happens in two parallel worlds—same disaster, same moment, same impossible pressure. In one world is Team Alpha. In the other, Team Beta. Both were deeply invested in the Futurama launch. Both now faced the identical situation: same bug, same pressure, same customers filing tickets. Neither team is fictional in the ways that matter—their behaviours, their instincts, and their choices are drawn from real patterns observed across real organisations.

Let's see how Team Alpha and Team Beta each react.

Team Alpha: Descenting into a Blame-Storm

Hour One — The First Move

Inside Team Alpha's conference room, the air turned toxic within seconds of the dashboards going flat. The team leader—a sharp, experienced manager who had invested enormous personal capital in this launch—felt the crisis not as a technical problem to be solved, but as a direct attack on his reputation. The 'image cost' was immediate and overwhelming. Something had to be done. Someone had to be held responsible— and it was already clear, in the leader's mind, that this someone would not be found anywhere near his own team.

The leader's first thought was not a real question. It was a verdict.

Not us. We followed the plan, ran every review, did everything right. This has to be something in their integration layer. This is on them.

That internal monologue took perhaps four seconds. What followed it took considerably longer to undo.

Everyone in the room saw it happen in real time. The leader's jaw tightened. His eyes moved across the frozen dashboard with the hard, scanning focus of someone looking not for a solution but for a target. A pen was slapped down on the table. The leader pushed back from his chair, stood up, and the room contracted.

"This is unbelievable." The voice was loud, tight, the kind of anger that doesn't bother looking for an exit. "We did everything right. Every review. Every deadline. Every gate. And now this?" A hand swept toward the dashboard. "That's not on us. The data we got from the stakeholders was garbage from day one—I said it then and nobody listened. This is *their* mess. Not ours."

No one spoke. The verdict had been delivered before a single log had been opened, before a single hypothesis had been formed. The case was closed. The room just hadn't been told yet that the investigation was already over.

The outburst had done its work with brutal efficiency: every person in the room had just received the same unspoken instruction, and they had all understood it perfectly. One engineer glanced at their laptop screen and then, almost imperceptibly, angled it slightly away from the leader—an instinctive, physical act of self-protection. Another reached for their coffee mug and found it was already empty; they held it anyway, needing something to do with their hands. Eyes dropped. Shoulders tightened. A junior developer who had been about to open her mouth—she had a hypothesis, a thread worth discussing—closed it again. She would revisit that hypothesis quietly, privately, later. Not now. Not in this room. Not with him in this state.

The most experienced engineers in the room—the people best positioned to actually solve the problem—had gone completely silent. The leader had, in under two minutes, made it more dangerous to speak than to say nothing.

The 'I Told You So' Trap

With the leader's posture set to defence, the rest of the room followed. Instead of pulling up logs and debugging tools, conversations fractured into something uglier. A senior engineer leaned back in his chair with a grim satisfaction.

"I told you we shouldn't have relied on their API." A pause. "We should have just built this ourselves from the start."

Others nodded. Others chimed in. The room began to fill with retrospective wisdom—the kind that is always cheapest after a failure and most useless during one. What no one mentioned: three days earlier, he had flagged exactly this integration concern in a private message to the team—and then let it drop, unwilling to risk derailing the launch.

This is perhaps the cruellest irony of a blame-driven culture: the person who holds the key to solving the problem is often the last one to speak, because the cost of admitting "I saw something and didn't say it" is too high when the room is already looking for someone to punish.

> *"In virtually every crisis, someone in the room already knows something useful. The question is whether they feel safe enough to say it."*

The Email to the Boss

By the end of Day One, the feature was still broken and customers were still affected— but Team Alpha's leader had shifted from managing the crisis to managing the narrative. Drafting a message to their boss required evidence, and gathering evidence required people: one pulling meeting timelines, another extracting sign-off records, a third spending two hours logging every partner data handoff, annotated for anything that could be read as a warning sign in hindsight.

While the pipeline bled, the team's collective energy was directed not at the bug but at the story to tell. Every screen that should have been running diagnostic tools was instead open on shared documents, everyone quietly building the case that the fault lay elsewhere. The actual problem—the broken system that real customers were hitting in real time—was, for those hours, nobody's primary task.

In that moment, something subtle but significant had shifted. The goal was no longer to fix what was broken, but to ensure that what was visible looked defensible.

This is how Potemkin[2] organizations emerge: through a gradual reweighting of priorities, in which reputation crowds out reality. From the outside, everything looks right—metrics trend upward, documents are polished, narratives are coherent. But beneath that surface, systems are fragile, problems go unaddressed, and teams quietly struggle. The facade becomes the work. Documents replace diagnostics. Narratives replace action. What matters is no longer whether the system works, but whether the story about it does.

[2]"Potemkin facades" describes the practice of creating a polished outward appearance to mask internal weaknesses, a term derived from the legend of Grigory Potemkin staging fake villages for Catherine the Great.

> *"In a Potemkin organization, the story matters more than the work. They look perfect from the outside—and fragile everywhere else."*

The resulting message drafted by Team Alpha was a careful, three-paragraph construction that explained, in detail, why everything the team had done was correct, and why the failure should be attributed to the decisions and data quality of the partner teams.

It was, in its way, an impressive piece of writing. It was also a near-perfect expression of the Potemkin instinct to polish the front while the structure behind it weakens, but also of a strong victim's mindset.

This is the mindset that takes hold when a leader—or a team—stops asking "what's my part in this, and what can I do about it?" and starts asking "who do I blame for this?" It is seductive precisely because it is easy, and because it can feel entirely justified. Many of the explanations Team Alpha offered were probably accurate. The partner team's data quality may well have been poor. The integration process may genuinely have had gaps that others owned.

But here is the uncomfortable truth that both the victim's mindset and the Potemkin instinct obscure: truth and usefulness are not the same thing. Every accurate explanation in that email was useless. Not because it was wrong, but because it was pointed in the wrong direction. It looked backward, at blame, when the only direction that could help anyone was forward, at the fix.

And there was another truth buried in that email, unacknowledged. In virtually every failure of this kind, everyone involved has played a role—even if that role was simply staying silent when something felt wrong, or not asking the question that might have delayed the launch

by a day but caught the bug before it reached customers. Team Alpha's leader had made choices too. So had everyone who gathered data for the brief instead of running diagnostics. Recognizing this is not self-blame. It is the only honest starting point for solving anything.

> *"A victim's mindset confuses being right with being useful. You may be right—but in a crisis, blame won't help, and thinking like a victim never will."*

The message contained no proposals for a fix. It contained no timeline for resolution. It contained no ask—no resources requested, no escalation sought. It closed with the suggestion that the organization should "consider evaluating the reliability of partner teams."

The leader sent it, felt briefly better, and returned to the conference room, where the dashboards were still red.

And in that room sat the quiet hallmark of a Potemkin organization: everything looked under control—except the reality that mattered.

Days Two Through Five — The Broken Flywheel

The situation with the partner teams deteriorated rapidly. When stakeholders from those teams attempted to join the debugging sessions—genuinely trying to help, bringing their own logs and hypotheses—they were met with barely concealed hostility. Team Alpha's leadership team frequently interrupted with pointed questions designed less to gather information than to establish a record of where the fault lay.

Arguments grew loud. Voices were raised in meetings and on video calls. One partner team manager, feeling publicly blamed for a failure

she wasn't convinced her team had caused, stopped responding to pings altogether. Another team pulled their engineers out of the joint debugging channel. The collaborative network that had built Futurama over six months collapsed within seventy-two hours.

What had been destroyed in those seventy-two hours was not just a working relationship. It was trust—the kind that had been built slowly and quietly over six months of shared work, late-night debugging sessions, and the hundreds of small moments of reliability that accumulate into the belief that the person on the other side of the table is on your side. That trust was now gone. And everyone in every team involved understood, at some level, that it would not come back quickly—if it came back at all.

> *"It takes a lifetime to earn someone's trust. But one thoughtless act is enough to break it beyond repair."*

By Day Five, the feature remained broken, the pipeline was still failing customers, and no one—not in Team Alpha, not in any of the partner teams—had yet identified the actual root cause. Every hour had been directed at reconstructing a timeline of blame rather than a map of solutions. Teams were looking backward with forensic intensity at who had made which decision, while the bug continued to bleed.

They had successfully navigated themselves right into a trap of their own making: technically capable, deeply motivated, and completely unable to help anyone. The victim's mentality had not just failed to solve the problem. It had made the problem structurally harder to solve, by dismantling the very network of trust that solving it required.

Team Beta: Ascending into a Vulnerability Loop

Hour One — A Different First Move

In Team Beta's conference room, the situation was identical—same red dashboards, same sick feeling in the stomach, same awareness that something had gone very wrong. For a moment, the leader of Team Beta felt exactly what her counterpart in Team Alpha's room felt: the sharp, instinctive pull toward blame. Six months of work. Two slipped dates. And now this. The image cost was immediate and real.

She noticed the feeling. And she made a different choice.

She didn't look for a scapegoat. She looked at the room.

"We are all in the same boat," she said, and she meant it. "I didn't ensure the integration vetting was rigorous enough before we pushed. I should have pushed harder on the final API checks. That's on me." A breath. "Now—let's not spend another second on what happened. Let's figure out what's broken, fix it, and then we can do a proper review of how we got here. Does anyone have an idea on where to start?"

The silence that followed was nothing like the silence in Team Alpha's room. It lasted perhaps two seconds. Then a laptop lid opened. Someone pulled up a terminal. A senior engineer leaned forward, elbows on the table. The body language of the room rearranged itself—from braced-for-impact to ready-to-work.

No one had been told it was safe to think aloud. They had been shown.

The Human Sensor Speaks Up

Because the leader had established, in thirty seconds, that the room was a safe place for uncomfortable truths, something happened that almost certainly would never have occurred in Team Alpha's conference room.

A junior engineer on the team—quiet, technically meticulous, the kind of person who notices things in logs that others scroll past—raised his hand.

"I think I might know something." A pause. "A few days ago I noticed an anomaly in the API response times during the pre-launch testing window. I meant to flag it, but the launch window was so compressed and everything else seemed fine, and I—" He stopped. "I'm sorry. I forgot. Or maybe I talked myself out of it. I'm not sure."

The room waited. Team Beta's leader looked at him for a moment.

"Thank you," she said. "Genuinely. That's exactly what we need right now. Walk us through what you saw."

No flash of temper. No pointed silence. No implication that this admission was a confession of guilt rather than a contribution to a solution. The appreciation was immediate, specific, and real—and it sent a signal to every other person in the room: if you know something, say it. There is no cost here for honesty.

> *"The courage to say 'I noticed something before but didn't speak up' is the most valuable thing in a crisis. Punishing it is one of the most expensive mistakes."*

What made this moment possible was not luck, and it was not the junior engineer's unusual bravery. It was a principle Team Beta's leader had worked to embed long before the crisis: *everyone is a sensor*.

Every individual on a team—regardless of title or tenure—is a sensor. Each person perceives the world from a slightly different vantage point, shaped by their experiences, their instincts, and their blind spots. A data scientist notices something in the response curves that the infrastructure engineer scrolls past. A product manager catches an implication in customer behaviour that the analyst didn't flag. A junior team member, still new enough to find things surprising, spots an anomaly the senior engineers have unconsciously normalised. These are not failures of the senior team—they are the natural consequence of diversity of perspective. But they only become useful if the person sensing them feels both entitled and safe enough to say so.

In many teams, critical signals go unprocessed not because no one noticed them, but because the person who noticed them didn't believe it was their place to speak—or had learned, through past experience, that speaking came with a cost. A team's sensors are only as good as its channels for processing what they detect.

> *"Every person on a team is a sensor. Critical signals rarely go unnoticed —they just usually go unheard."*

In practice, what these sensors pick up are not just issues, but tensions—early signals of mismatches, improvement opportunities, and unrealised potential. Think of a tension as the gap between the current reality and a better one you can sense but haven't yet reached. Every person on the team generates them. Most organisations leave them unprocessed, diffusing quietly as frustration or disengagement, never making it into a conversation that could turn them into change.

When every person has the authority and the obligation to name what they are sensing, and the organisation has a reliable mechanism

to do something with it. When such a system is in place, you have a reasonable chance that a junior engineer who noticed an anomaly three days before a catastrophic launch will say so. When it is not, they talk themselves out of it—and the anomaly waits.

The Collaborative Fix

Within the hour, the junior engineer's observation had become the team's primary thread of investigation. Team Beta reached out to partner stakeholders—not to assign blame or demand answers, but to request their help, explicitly acknowledging their unique expertise and what only they could bring to the problem. The message was clear: "we think we found something and we need your expertise." Those partner engineers joined the discussion, felt the ask was real, and brought their own data.

The atmosphere was tense. Psychological safety has nothing to do with comfort— it means that discomfort can be expressed and worked with rather than hidden. People were stressed, tired, and anxious. But they were also honest, and that is what the situation required.

Ten hours after the dashboards turned red, Team Beta found the root cause: a subtle but critical timing mismatch in the API handshake that only manifested under the specific combination of production load and the new front-end rendering logic. Under that load, the mismatch didn't just suppress the new feature—it corrupted the response handling for a portion of the existing pipeline, causing failures for a huge number of customers. It had never appeared in testing because it required real traffic volumes to trigger. It was, in retrospect, almost elegant in its obscurity.

The fix took forty minutes to implement, test, and deploy.

When the feature went live again and the dashboards finally started to move in the right direction, someone opened a bottle of wine. Someone else ordered pizza. The celebration was hard-won—not the pre-planned champagne of a launch, but the scrappier, warmer satisfaction of a problem solved together under real pressure.

But something else had happened in those ten hours that no one had planned for. The people trying to find a solution—from Team Beta, from the partner teams, from corners of the organisation who had never had reason to work closely together—had gotten to know each other. Not in the polite, corporate-ritual way of a company all-hands, but in the way that only shared adversity produces. They had seen each other think under pressure, admit uncertainty, change their minds, and contribute ideas that turned out to matter. People had learned, in real time, what each team actually cared about, what constraints they were operating under, where their expertise actually lay—and where it didn't. The organisational chart is a diagram. This was the real map.

> *"There is no team-building exercise more powerful than a real problem that requires real people to be honest with each other."*

By the end of it, people who had been strangers in the morning were sending each other direct messages that had nothing to do with the fix. New working relationships had formed—the kind built on the specific, durable foundation of having solved something hard together. Some of them would become lasting friendships.

No party would have produced this. No offsite, no team-building exercise, no carefully facilitated workshop on cross-functional collab-

oration. Those events have their place, but they cannot manufacture what ten hours of honest, high-stakes problem-solving creates naturally. The crisis, handled the right way, had done more for the connective tissue of the organisation than a year of corporate calendar events.

Days Two Through Five — The Flywheel Starts Turning

The days that followed felt different from anything that had come before the launch. The immediate pressure was gone, but the energy it had produced did not dissipate—it redirected. People who had worked side by side in the debugging channel stayed in contact. Questions that would previously have required a formal meeting request were now sent directly, informally, and answered quickly. The invisible friction that accumulates between teams who don't really know each other—the hesitation before reaching out, the careful, formal framing of every ask—had been significantly reduced. Small things changed in ways that were hard to attribute to any single decision. A design review that might have run without the right technical input was flagged by someone who now knew exactly who to loop in. Decisions that had previously required escalation were now happening at the working level, faster and with more context.

Something else emerged from the collision of perspectives the crisis had forced together. With people from different disciplines and teams all thinking aloud about the same problem, diverse viewpoints met on the ground in a way that formal planning processes rarely produce. Multiple new feature ideas surfaced organically—observations about customer behaviour, gaps in the existing product, and possibilities that no single team would have seen alone.

And the team's collective instinct for risk had sharpened. People who had just lived through one failure started asking: where else could

this happen? Potential issues in adjacent systems were identified, mitigation strategies developed, and fixes scoped and prioritised—all before any error had occurred.

> *"A shared crisis leaves not just scars, but connections—fragile in the moment, but often the source of the next breakthrough or averted disaster."*

None of this appeared in any project plan. It was the residue of the crisis, distributed quietly through the organisation in the form of relationships that hadn't existed five days earlier. Team Beta had not just fixed a bug. They had, inadvertently, rewired a small but meaningful portion of how the organisation actually functioned.

By Day Five, the team was not recovering from the launch. They were, in almost every measurable sense, stronger than they had been before it.

The Transparent Reckoning

The real mark of Team Beta's culture, however, wasn't visible in the conference room during the crisis. It appeared in the days that followed.

Rather than quietly burying the incident and moving on, the team did something that many organisations find surprisingly difficult: they shared what had happened—openly, in detail, and across the whole company. Not as a confession, and not as a defensive exercise in distributing blame. As a gift.

The API timing mismatch was framed not as a mistake to be ashamed of, but as an intelligent failure: a gap in the team's collective knowledge that had been filled, painfully but permanently, by the

experience of the launch.

This framing mattered more than it might appear. There is a profound difference between treating a mistake as evidence of inadequacy and treating it as a signal that your understanding of the system was incomplete. The first response produces shame, concealment, and the desperate urge to assign blame elsewhere. The second produces curiosity—a real desire to understand what the failure was trying to tell you. Team Beta's leader had modelled this from the first minutes of the crisis. The write-up that followed institutionalised it.

The underlying belief, stated or not, was this: mistakes are not the opposite of competence. In any organisation doing genuinely innovative work, they are *necessary*—how a team discovers the edges of its knowledge, the places where its model of the world doesn't quite match the world itself. A team that never makes mistakes has stopped pushing into new territory. A team that finishes the year with every goal marked green should not be automatically celebrated—it should raise a question: were the goals ambitious enough? A perfect scorecard can be a red flag, a signal that the team optimised for safety rather than invention—for targets that were comfortable precisely because they required execution, not invention. The goal is not to eliminate failure. It is to approach it with enough curiosity to extract everything it has to teach.

Curiosity, in this context, is a choice—not a personality trait—and one that becomes easier with practice and harder under pressure, which is precisely why the mechanisms that support it need to be built in advance. When Team Beta's leader asked "what can we do right now?" instead of "who is responsible?", she was not naturally more curious than Team Alpha's leader. She had simply built the habit of asking the more useful question.

> *"Making mistakes is the only way to truly learn. If you only set goals you know you can hit, you are unlikely to fail—but you are equally unlikely to learn."*

What the team shared was read widely. Three other teams identified similar potential vulnerabilities in their own systems and preemptively addressed them. The failure of the Futurama launch, distributed as knowledge rather than buried as embarrassment, made the entire organisation incrementally stronger.

Team Beta turned a catastrophic launch into a permanent upgrade of the organisation's collective intelligence. And they did it not by being exceptional, but by being curious.

The Mechanisms That Make the Difference

The divergence between Team Alpha and Team Beta was not a story about talent, experience, or resources. The people on both teams were highly capable. The leaders on both teams cared about outcomes. The difference—the entire difference—was in the culture and mechanisms each team had built (or failed to build) for handling the moment when things go wrong.

Culture is not what you say in an all-hands meeting. It is not the leadership principles written on the wall of the lobby. It is not the PR announcement declaring what a great place to work you have built, or the carefully worded paragraph in the company handbook about psychological safety and inclusion. Culture is the aggregate of every small behavioral and communication choice made, especially when the pres-

sure is on: how a leader responds when someone delivers bad news, whether an engineer who noticed something feels safe enough to say so, whether the first question in a crisis is "who's responsible?" or "what can we do right now?"

Those small choices compound. And they spread—because humans are wired to mirror one another. Researchers call it behavioural contagion: we observe the people around us, absorb what they do, and unconsciously replicate it. This is how social norms form—not through policy, but through observed behaviour repeated often enough to start feeling like what *people like us do here*. One person who consistently models the right behaviour shifts the calculus for everyone watching. Enough repetitions, and it tips from "something she does" to "something we do." Anyone can start the contagion.

The same mechanism works with equal efficiency in the opposite direction. Defensive cultures are not designed—they accumulate, one unremarked behaviour at a time. Both conference rooms on the day of the Futurama launch proved it: one leader's words set a room on fire with blame, the other's set it to work. Good and bad micro-behaviours do not stay contained—they amplify. They become habits. The habits become the organisation.

> *"Culture is not declared. It is caught. The behaviours people observe often enough become the behaviours they perform—whether those behaviours belong to the best version of the team, or the worst."*

Here, then, are some ideas for specific mechanisms—the concrete, repeatable practices—that might help you to get closer to Betas, than being Alphas.

1. Praising the Messenger

The single highest-leverage behaviour available to a leader is the way they respond when someone brings them a problem. Not the problem itself—the response to being told about it.

When Team Beta's leader thanked the junior engineer for his admission, she wasn't just being kind. She was programming the room. Every person watching that exchange updated their internal model of what was safe to say in this team. The cost of honesty dropped. The value of information rose. The room became, in that moment, more intelligent.

Team Alpha's leader did the opposite, raising their voice and looking for a scapegoat. The implicit message—that bad news would be weaponised—was sufficient to ensure that the person who held the key to the root cause kept it to themselves.

None of this is easy in the moment. In any crisis, emotions run high—and the instinct to react, to find someone to hold responsible, is deeply human. Most of those reactions happen before conscious thought catches up. The gap between the stimulus and the response is where leadership either shows up or doesn't.

Breathing helps. A deliberate pause—even a few seconds—between the moment you receive bad news and the moment you respond to it can be the difference between a reaction that shuts the room down and one that opens it up. This is not a metaphor. It is a physical interruption of an automatic process. Surprisingly, practices like meditation and mindfulness are directly relevant here—not as wellness habits, but as training for exactly these moments. The ability to notice an emotional reaction without immediately acting on it, to find the brief gap between feeling and response, is a skill that can be built. Crisis situations are not where you develop it. They are where you use it.

Praising the messenger is a high-performance information-gathering mechanism, not a soft, feel-good practice. Leaders who shoot messengers are not merely unkind; they are strategically blind.

2. *Mistake of the Week*

Psychological safety cannot be declared into existence. It has to be built, brick by brick, through repeated demonstrations that vulnerability is not punished.

One effective way to build this foundation is the Mistake of the Week ritual: a standing agenda item in any recurring team meeting—a standup, a weekly progress review, a team sync—where any team member, including the leader, can share a recent error and the specific lesson extracted from it.

Most recurring meetings default to progress updates and wins. That instinct is understandable, but it quietly signals that only successes are worth discussing—and leaves the team's most valuable learning material unprocessed on the floor. Adding a dedicated slot for mistakes rebalances that signal.

The goal is not confession or catharsis. The goal is normalisation. When even the most senior person in the room regularly and calmly describes their own recent failures, the implicit message to the team is: errors are information, not crimes. This is the organisation we all want to have. You are safe here.

Over time, this ritual produces a team that surfaces problems early—before they become crises—because the mechanism of surfacing has been practiced and de-stigmatised.

3. Failure Celebrations

Innovation and failure are inseparable. A team that never fails is a team that is not pushing hard enough into uncharted territory. The organisations that understand this build explicit rituals to celebrate intelligent failures—ambitious bets that didn't land, experiments that produced valuable negative results, projects that ended without the hoped-for outcome but generated learning that informed the next attempt.

When a project closes without its desired result, hold a celebration. Not a consolation party, not a post-mortem disguised as an event—an honest acknowledgment that the team took a real risk in pursuit of something worth pursuing, and that the failure itself has value. The centrepiece of that celebration should be the learnings: what the team now knows that it did not know before, what those lessons mean for the next attempt, and how the organisation is collectively smarter for having run the experiment. Raising and appreciating those learnings explicitly—naming them, writing them down, sharing them—is what transforms a failure from a loss into an asset.

But there is a problem that sits even upstream of celebration: in many organisations, failures are not celebrated because they are barely acknowledged. When a project quietly winds down without its goals met, the most common response is silence—a collective agreement to move on without naming what happened. No retrospective, no shared reflection, no honest accounting of what went wrong and why. The absence of celebration is only part of it. The deeper issue is the absence of acknowledgment itself. A culture that cannot name its failures cannot learn from them.

This is not sentimentality. It is strategic. Organisations that penalise failure— even implicitly, through the absence of celebration—train their people to take smaller and smaller bets. Over time, those

organisations become safe and slow. They enter the Day 2 trap[3]: stable, process-driven, and incrementally irrelevant.

4. Praising Effort Over Outcomes

The most dangerous time to shape a team's culture is when things are going well. In moments of success, it is tempting to praise the outcome—the number hit, the sales increased, the launch completed, the deal closed. This feels natural, even generous. But it quietly trains the team to optimise for outcomes, including by taking shortcuts, suppressing bad news, and avoiding the risks that look like they might not pay off.

The more powerful practice is to praise the inputs: the rigour of the analysis, the cross-org collaboration, the knowledge sharing, the courage of the flag that was raised. When a team member writes an exceptionally thorough design review— regardless of whether the project ultimately succeeds—that behaviour should be named, specifically and publicly, as the kind of work that defines the team.

The difference is concrete and sayable. "Great result" is outcome praise. "The way you pushed back on the timeline when the integration risk wasn't resolved, and wrote up the concern before anyone asked—that's the kind of judgment that makes this team better" is effort praise. The first trains people to deliver results. The second trains people to work with integrity, even when the result is uncertain.

The long-term effect of praising only outcomes is a team that learns to protect outcomes at any cost. Targets get sandbagged so they can

[3]The *Day 1 / Day 2* concept comes from Jeff Bezos's 2016 shareholder letter. Day 1 reflects a startup mindset—fast, customer-focused, and experimental— while Day 2 marks the drift into process, bureaucracy, and eventual decline. Staying in Day 1 requires deliberate effort; it is not the default.

be hit. Bad news gets managed rather than surfaced. Risks get taken quietly rather than flagged, because flagging them might delay the result everyone is waiting to be praised for. The Futurama crisis was, in part, a product of exactly this dynamic: a launch window that had become more important than the uncomfortable integration question no one raised.

This shifts the team's attention from what happened to how we work. It also sends a signal to the person who took the careful, rigorous path that ended in failure: that the work still mattered, that they are still valued, that the organisation is paying attention to the right things. A team that believes that is far more likely to do the right thing next time—whether or not it pays off.

5. *Zero Tolerance for Destructive Sarcasm*

There is a specific kind of toxin that corrodes psychological safety more quickly than almost anything else, and it is particularly insidious because it masquerades as wit: sarcasm directed at someone who has admitted ignorance, surfaced a problem, showed a knowledge gap or asked a "basic" question.

"Oh, you didn't know that?" "I thought we covered this three weeks ago." "Classic." These comments, delivered with a raised eyebrow or a dry smile, seem minor. They are not minor. Every person in earshot updates their internal model: this is what happens if you admit you don't know something. This is what it costs to ask a question. The silence that follows is not respect—it is self-protection.

Leaders who tolerate sarcasm are not choosing not to intervene in a trivial social dynamic. They are choosing to allow a slow but steady erosion of their team's willingness to communicate.

The response to sarcasm should be immediate and simple: name it, redirect it, and make the expectations clear. "Hey—we don't do that here. What's the actual question?" This takes six seconds and costs almost nothing. Not doing it costs the team, gradually and invisibly, almost everything.

The most powerful thing a leader can do in this space is model the behaviour themselves. Few things signal safety more clearly than a leader saying, without hesitation, "I don't know." A leader who admits a knowledge gap invites honesty. A leader who cannot admit one trains their team to perform certainty they do not have.

And this is not only the leader's responsibility. When a sarcastic comment lands and nobody says anything, the silence is a choice—made by every person in the room. Stepping up, calling out behaviour that erodes the team's culture, is not reserved for the most senior person present or the person with the biggest title. It is an expectation held of everyone. A team that waits for the boss to set the tone has outsourced something it should own collectively.

6. The Kaizen Town Hall

The Kaizen Town Hall is a regular, dedicated meeting—separate from delivery rituals, sprint reviews, and status updates—whose sole purpose is making the team better. Its raw material is what every person on the team is already noticing: friction points, improvement opportunities, things that could work better. In most organisations these diffuse quietly as frustration or disengagement, never making it into a conversation that could act on them. The Kaizen Town Hall is the mechanism that refuses to let that happen.

Anyone can put a topic on the agenda. The format is structured: the person raising an issue comes with a concrete proposal, grounded

in a specific example of how things could be better. The group works through it using a disciplined process— clarification, reactions, adaptation, and a concrete next step—designed to reach a decision quickly. The default is to lean in and try things: when in doubt, run an experiment. This sends a signal that proposals can actually change things—that raising an issue is not an exercise in venting, but an act with consequences.

But beyond the mechanics, the Kaizen Town Hall sends a signal just as important as anything discussed in it: that every person on the team, regardless of tenure or seniority, has both the standing and the expectation to drive meaningful change. Not just permission—expectation. Raising issues is not an invitation extended to those who feel like it. It is an expectation held of everyone. The implicit message is clear: improving how we work together is not the leader's job. It is everyone's.

7. Write It Down. Make It Searchable.

After any significant failure, the instinct in most organisations is to move on quickly and quietly. The mechanism here is a deliberate refusal to do that: write down what happened, why it happened, and what will prevent it from happening again —and make that document accessible to everyone in the organisation.

The act of writing forces a clarity that debrief conversations rarely achieve. Vague impressions become specific claims. Hunches become hypotheses. What felt like bad luck starts to look like a predictable consequence of specific decisions—which means it can be prevented next time. The discipline is not bureaucracy; it is the difference between knowledge that lives in a handful of people's heads and knowledge that belongs to the organisation.

The second requirement is equally important: the document must

be findable. A write-up filed away in a folder no one opens is no better than silence. The goal is a shared, searchable record that anyone—including someone who joined the team six months later—can find when they are about to make a similar decision. Most of the mistakes organisations repeat are not repeated out of carelessness. They are repeated because the people making them simply did not know the mistake had been made before.

One well-known example of this done deliberately is Amazon's Correction of Errors[4] (CoE)—a structured post-incident document, shared broadly and treated as an instrument of learning rather than accountability. The specific format matters less than the discipline: write it, share it, and make it searchable. A failure, processed well and made visible, stops being a loss. It becomes an asset.

The Choice That Defines the Culture

The Futurama story ends in two very different places. Team Alpha spent five days in a loop of blame and defensiveness, the bug unresolved, the existing pipeline still broken for customers, relationships fractured, trust destroyed. The talented people on that team began, quietly, to look elsewhere.

Team Beta spent ten hours solving the problem together in a collaborative manner, wrote a document that made the whole organisation smarter, held a spontaneous celebration that deepened the team's cohesion, and began the next project with more trust, passion and excitement—not less—than they'd had before.

[4] At Amazon, a *Correction of Errors* (CoE) is a post-incident document written after any significant failure, shared across the organisation and treated as an instrument of learning, not accountability.

Same crisis. Different choices. Entirely different futures.

The most important insight here is not that Team Beta was composed of better people, or that their leader was unusually gifted. The insight is that the mechanisms were in place *before* the crisis hit. The Mistake of the Week ritual had built the habit of surfacing errors without shame. The Kaizen Town Hall had given every person on the team a practiced sense that their observations could change things—so when the junior engineer noticed something, he believed it was worth saying. The leader's established pattern of appreciating honesty meant that his admission came out in the first hour rather than the fifth day—or not at all. And the practice of writing a CoE meant that when the crisis was over, the team had a container for the failure—a way to transform it into knowledge rather than let it quietly calcify into anxiety.

Good intentions don't build great cultures. Mechanisms do.

We are living in a VUCA world—Volatile, Uncertain, Complex, and Ambiguous— and the pace is not stabilising. It is accelerating. The AI era will not reduce the frequency of complex failures. If anything, it will increase them. The systems we are building are more powerful, more interdependent, and more opaque than anything that came before. The gap between what we intend and what actually happens will remain wide, and will occasionally be traversed by disasters we didn't anticipate. The organisations that navigate this era well will not be the ones that somehow avoid failure—they will be the ones that have built the culture to face it.

What we can control is not whether those disasters happen. What we can control is the culture that greets them—the habits, the rituals, the daily choices that determine whether our teams run toward the problem or away from it, whether the person who knows something

says it or stays quiet, whether failure becomes information or becomes shame.

That culture is built one choice at a time. It starts with how you respond, today, the next time someone approaches you with bad news.

Whisper the wins. Shout the mistakes. Build the mechanisms.

The rest will follow.

Chapter Summary: Blame vs. Execute

	Team Alpha *The Blame-Storm*	**Team Beta** *The Vulnerability Loop*
First Question	*"Who is responsible for this mess?"*	*"What can we do right now?"*
Mindset	Backward-looking: assign blame	Forward-looking: find the fix
Response to Error	Victim's Mindset — protect image	Vulnerability Loop — protect customer
Communication	Finger-pointing & dangerous silence	Honest, forward-focused, curiosity-driven
Key Mechanism	Defensive emails & blame timelines	Praising the messenger
Time to Resolution	5+ days — root cause never found; pipeline still broken	10 hours — pipeline restored, feature live
Long-Term Outcome	Trust destroyed; talent leaving	Stronger teams, smarter organisation

Seven Mechanisms for a Solution Culture

1. Praising the Messenger

Situation — Someone brings bad news, flags a problem, or admits a mistake—especially under pressure.

Behaviour

→ First response must be appreciation—specific and immediate

→ Never lead with analysis or consequence

→ If an emotional reaction rises, pause before speaking

→ Ask one question before making any statement

Impact — Every person watching updates their sense of what's safe to say. Bad news travels faster. The person who holds the key to the root cause speaks up instead of going quiet.

2. Mistake of the Week

Situation — Any recurring team meeting—standup, weekly sync, or progress review.

Behaviour

→ Add a standing slot for mistakes—leader goes first, every time

→ Share one recent error and the specific lesson extracted

→ If no one volunteers, the leader shares one immediately

Impact — Normalisation requires repetition. A team that practises surfacing errors surfaces them before they become crises.

3. Failure Celebrations

Situation — A project closes without its desired outcome.

Behaviour

→ Hold a celebration within two weeks—centre it on learnings

→ Name what the team now knows that it didn't before; write it down and share it

→ If the team moves on without acknowledging what happened: stop, name it, share it

Impact — Teams continue taking ambitious bets. Failure is separated from shame. Organisations that skip this step enter the Day 2 trap—safe, slow, and incrementally irrelevant.

4. Praising Effort Over Outcomes

Situation — A team member demonstrates high-quality process—rigorous analysis, a proactive flag, strong collaboration—regardless of outcome.

Behaviour

→ Name the behaviour specifically: the rigour, the flag raised, the collaboration

→ Decouple praise from whether the outcome succeeded

→ If a project fails despite excellent process, praise the process publicly anyway

Impact — What gets praised gets repeated. Praising process trains quality—and quality, compounded over years, determines everything else.

5. Zero Tolerance for Destructive Sarcasm

Situation — A sarcastic comment targets ignorance, a question, or an honest admission.

Behaviour

→ Intervene immediately: "We don't do that here"

→ Redirect to the substance: "What's the actual question?"

→ This applies to everyone—not only the leader

Impact — Every unchallenged sarcastic comment raises the cost of honesty for everyone in earshot. The silence that follows is not respect—it is self-protection.

6. The Kaizen Town Hall

Situation — Issues on the team are going unaddressed; improvement ideas have nowhere to go.

Behaviour

→ Schedule a dedicated session separate from delivery rituals—sole purpose: making the team better

→ Anyone raises an issue with a concrete proposal; session not done until a next step is agreed

→ When in doubt, run the experiment

→ Raising issues is an expectation held of everyone, not an invitation

Impact — Critical signals rarely go unnoticed— they go unheard. A reliable channel converts individual perception into collective intelligence. Improving how the team works is everyone's job.

7. Creating the Error Artefact

Situation — A significant failure or incident has occurred.

Behaviour

→ Write a structured document: what happened, why (Five Whys), what changes

→ Name decisions and processes—never people

→ Share it broadly, beyond the team involved; make it searchable

Impact — A failure processed well and shared widely stops being a loss. It becomes an asset—others learn the lesson without paying the cost of living through it themselves.